

Assembly is nearly free. So why does almost no one capture the value?



Weeks of work → hours ^[1]

Anthropic internal experience

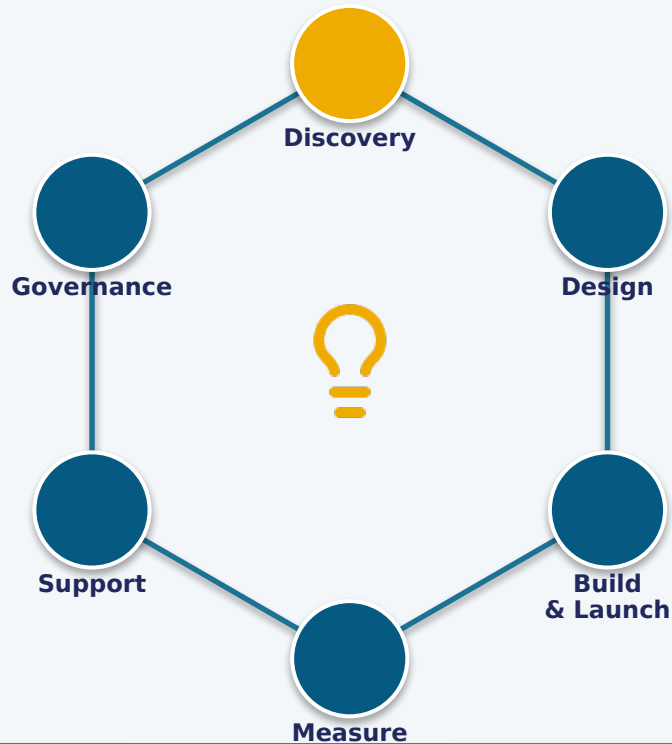


~95% of pilots — zero return ^[2]

MIT, summer 2025

Both facts are true at the same time: assembly became nearly free, but turning assembly into value did not. Hold the question — we'll come back to it at the end of the lecture.





LECTURE 5

AI Product: the full lifecycle — from intent to operation

Course "Deliberate Use of AI" · 3rd-year engineering students

Six sections — six arrows of one loop: code (Lecture 4) is one cheap step inside it.

0
Intro

1
Discovery

2
Design

3
Build & Launch

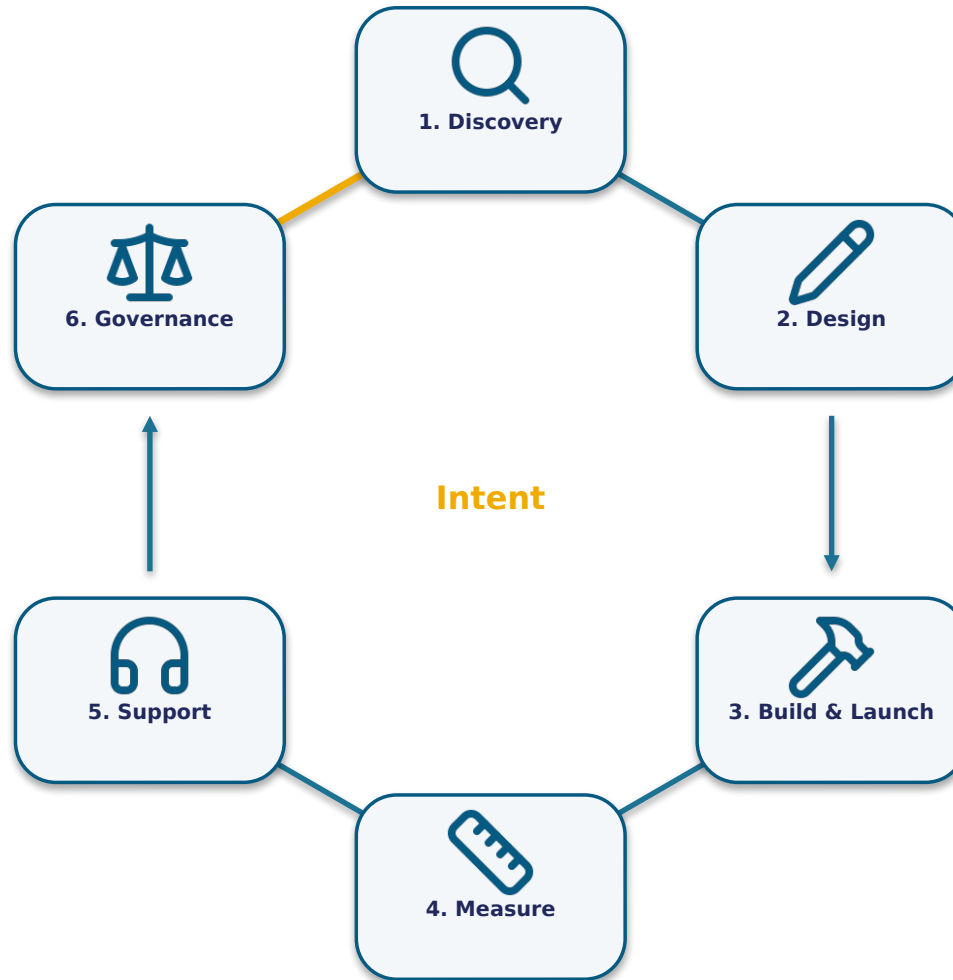
4
Measure

5
Support

6
Governance

7
Synthesis

The lecture is a loop, not a list of six topics



1. Discovery — where the hypothesis comes from

2. Design — hypothesis → artifact

3. Build & Launch — artifact → product

4. Measure — did it work

5. Support — lives 24/7

6. Governance — where to invest

The return arrow: governance feeds discovery again — a closed loop, not a one-time linear process.

Why a product needs the loop: without one, the error is found too late

Five questions a product answers by observation rather than by opinion

1 Does anyone need this at all

phase 1 · discovery

2 Can a person actually use it, and will someone nobody thought about be harmed

phase 2 · design

3 Can we build it and release it safely

phase 3 · build and launch

4 Will it be chosen firmly enough that people pay for it

phases 4 and 6 · measurement, governance

5 Do those answers still hold a quarter later

phase 5 · support

The first four are the product risks named by Marty Cagan (product leader, founder of the Silicon Valley Product Group, author of "Inspired"); the fifth only opens up after launch. Each has a phase of its own: the loop is the minimum set of places where these questions get answered.

When there is no loop

The questions do not disappear — optimistic assumptions take their place.

The error does not disappear either; only the moment of finding it moves — to after the build has been paid for and the release has happened, where rolling back costs more than the error itself.

From requirements to release, the cost of a fix spreads out up to a hundredfold on large projects and fourfold on small ones (Barry Boehm, a researcher in the economics of software development, 1981).

From inside the team it does not look like failure

Built it, nobody uses it · got to a pilot and stalled · it works and it is not chosen · rolled it out, and a quarter later people stopped opening it.

In all four cases the build itself went through successfully — which is why from inside the team it reads as success; an error sitting inside an unchecked hypothesis is not visible to anyone.

The expense of building worked as an involuntary barrier: while making the thing cost weeks, that cost alone forced teams to think in advance. AI is exactly what zeroed it out — and teams that never had an explicit loop now have nothing left holding them back from skipping these questions.

AI changes the cost and trust of every arrow of the loop — but not equally



Build

cost nearly collapsed to zero



Measure / Learn

cost about the same, trust fell



Observe / Orient

faster, but more exposed to attack

A meta-pattern repeating in each of the 6 sections: every arrow has a classical discipline — AI doesn't cancel it, it changes its cost and the degree of verification it requires.

Think about it: on which arrow of your latest task did speed outrun your actual trust in the result?

SECTION 1

Discovery — intent and hypothesis

2 base cases · 3 failures

Where does a hypothesis about what to build even come from. Most people have the informal experience of "ask your friends" — and almost no one has met the formal discipline that explains why that experience systematically lies.



0

Intro

1

Discovery

2

Design

3

Build & Launch

4

Measure

5

Support

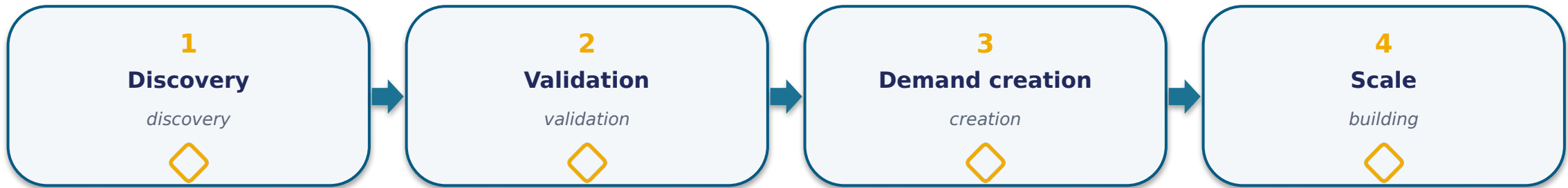
6

Governance

7

Synthesis

"There are no facts inside the building" — 4 steps, each ending in a decision



Steve Blank — the Customer Development methodology ^[1]



Falsifiable hypothesis

we believe X → we'll test it via Y → by date Z we'll have a yes/no answer

BML is the engine; Customer Development is the map of the terrain the engine drives on. First understand what you're testing and with whom — only then how fast to spin the loop.

^[1] [Blank — Customer Development \(Four Steps to the Epiphany\)](#)

Hours → minutes: but links exist so they can be checked



Desk research

Perplexity Deep Research: 2 hours → ~30 minutes ^[1]

verify the source



Interview synthesis

Dovetail — pain-point clustering at scale ^[2]



Reference dataset

a curated collection of "query → correct answer"

97% of researchers use AI — only ~8% trust AI personas as data ^[3]

Mandatory practice: verify important data directly against the source before using it in a decision. Synthetic users are only a pre-research step (piloting a guide, a draft of personas, generating hypotheses), never evidence for a go/no-go decision.

Rehearsing interview questions on AI personas

WHAT IT IS

Before going out to living people, the list of questions is run past several invented interviewees played by a model. The point is to catch bad questions before they spoil a real conversation. The run says nothing about the product. A rehearsal, not research.

HOW IT WORKS

- 1 Several different customer types are described — by how a person lives with the task: what they use now, what it costs them, what they once tried and dropped. **A character sketch is no use here.**
- 2 Each type is kept apart, so that the answers do not run together into one common voice.
- 3 The same list of questions is run past all of them.
- 4 Two things are read, **and the answers are not among them:** a question everyone answered the same way tells nobody apart; a question answered in terms of an imagined future invites a polite invention. Both get rewritten.



the invented interviewees do not hear each other



**the only thing that leaves is
a corrected list of questions**

WITH AI AND WITHOUT

WITHOUT AI — how the rehearsal has always been run

The list of questions is talked through with a colleague, or one live interview out of eight is spent on the rehearsal. It works, but it gives one point of view and costs a day.

WITH AI — what changed

The rehearsal takes minutes, and a dozen runs are affordable. **Only the rehearsal** gets cheaper: nothing replaces the interview with a living person.

WHERE IT BREAKS

The interviewees' answers are not data about users: there is one model, and five interviewees give five views of that one model. There are no five people behind them. Checked against a real national survey, the model reproduces the average answers and halves the spread of opinion (16 against 31). If a conclusion about the product has been carried out of the run, the practice has been run wrong: its output is a corrected question.

Searching accumulated support tickets for the pain

WHAT IT IS

A product with users already has an archive of complaints and questions. The practice turns it into a source of hypotheses: the tickets go into a searchable store, the ones close in meaning are grouped, and you see which themes repeat. What leaves is a named theme of pain.

HOW IT WORKS

- 1 Every ticket goes into one common store, tagged with when it arrived, who it came from and which product it is about.
- 2 The search runs **two ways at once**: on an exact match of words, and on meaning. An error code is found only by the first; a problem described in a person's own words only by the second.
- 3 What is close is gathered into groups and retold in two passes: first each ticket on its own, then what the group has in common. Skip the first pass and 20–40% of the detail is lost.

 **store and tag**

 **search by word and by meaning**

 **gather what is close into groups**

order matters: skip the tagging and the archive becomes a bag of text

WITH AI AND WITHOUT

WITHOUT AI — tagging by theme, by hand

Tickets are sorted into a list of themes thought up in advance. So the only themes found are the ones somebody has already guessed at.

WITH AI — what changed

What is close in meaning gathers itself, with no list of themes, and a group is retold in minutes. **The name of the theme is still checked by a person:** the machine groups, it takes no decisions.

WHERE IT BREAKS

The archive is biased by construction: it holds only those who got as far as complaining. Volume does not cure it: there is no ticket saying "I need what you do not have at all". The question "why do people not get as far as signing up" is not one it answers: that takes a conversation with those who left. And the search returns what is close, it does not weigh importance: ten thousand complaints about one typo are still one typo.

 **who got as far as complaining**

 *all users*

 **the ticket archive**

A synthetic interviewee structurally cannot produce disagreement



Mirror of agreement: a synthetic interviewee structurally cannot reflect disagreement — it has no real past that could contradict the way the question is phrased.

When a live interview is strictly better than AI synthesis



1. The cost of a "continue/stop" mistake is high



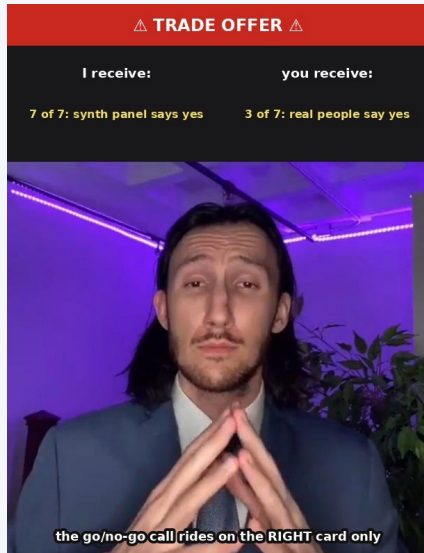
2. You need a real past, not a plausible one



3. You need validation by commitment, not by opinion

AI summaries lose 20-40% of interview detail (Torres) ^[1] when the "individually first" step is skipped — a failure traceable to a specific step, not a vague "AI is sometimes wrong."

The same task: real people — 3 of 7, the synthetic panel — 7 of 7 ^[1]



Real people



3 of 7 — "far-fetched and useless"

Synth panel



7 of 7 — "a game changer"

The same feature — opposite verdicts

Criterion: synthetic output is disqualified for a "continue/stop" decision by construction, not from bad luck on a run. Alternative: a pre-research role + real testing with 5-8 participants.

10,000 devices sold out of the 100,000 planned: the comparison with the phone in the buyer's pocket was made after the build



WHAT HAPPENED Humane is a company founded in 2018 by people who came out of Apple. Its only product, the AI Pin, is a wearable device with no screen: a small box worn on your clothes with a camera, a microphone and a laser projection onto your palm, answering out loud through a language model. The idea: a person stops looking at their phone. The device was shown on 9 November 2023 and sales opened in April 2024 — \$699 plus \$24 a month. On 18 February 2025 HP bought the assets for \$116 million, and on 28 February the cloud service was shut down: the devices in buyers' hands stopped answering.



What the numbers showed

- About 10,000 devices sold — a tenth of the company's own target of 100,000 by the end of the year
- Revenue of about \$9 million against about \$230 million raised — roughly 4%
- Between May and August 2024, more returns than purchases: by August, closer to 7,000 were still in people's hands
- The price was cut from \$699 to \$499 on 23 October 2024 — six months after sales opened



What was not found out before the build

The discovery phase's question fits in one phrase: **compared with what**. Every buyer already had a phone in their pocket that does the same things, and everything else on top. Reviewers converged on the same point: the device does less than a phone and does it more slowly.

An answer like that costs eight conversations with living people before the build. Here it arrived from the market — by way of \$230 million and 10.5 months of sales.

The answer about the user arrives either way; the discovery phase only chooses when, and at what price. The criterion that carries forward: a hypothesis about the user is closed by comparing it with whatever that person gets by with today.

The device does work, and the miss lies in the answer to who needs it and what for. Google Glass travelled the same road in the opposite direction: the \$1,500 glasses were pulled from general sale in January 2015 and then handed to people whose hands are busy with work. At AGCO, a maker of agricultural machinery, machine assembly time fell by 25% and inspection time by 30%; at the logistics company DHL, warehouse output rose by 15% — all against the same work done without the glasses. Same device. Different user.

SECTION 2

Design — a hypothesis becomes an artifact

2 base cases • 2 failures

How a hypothesis from research turns into a concrete artifact you can show a person. This is also where safety gets built in — in the design brief, not patched on after something has already gone wrong.



jump straight to the solution

the right problem first

0

Intro

1

Discovery

2

Design

3

Build & Launch

4

Measure

5

Support

6

Governance

7

Synthesis

Design here is the whole of a person's work with the system: surfaces and touchpoints, behaviour, the answer when unsure



THE SCOPE OF THE WORD The screen is one of the surfaces on which a person meets the system. Design in this phase settles the whole make-up of that meeting: where the system comes across a person, how it behaves over time, and what it does in the minute when it is unsure of its own answer. In a product built on a language model, there is usually more work on the other surfaces than there is on the screen.

Surfaces and touchpoints



Screen

what a person sees, and in what order



Voice

the same decision with no picture: order, length, the right to interrupt



Notification

the system opens the conversation itself: when, and on what occasion



Email, report

the answer read when the system is nowhere at hand



Refusal

what the system says when it will not do the thing



Silence

processing is running, there is no answer yet: what a person sees for that minute

The system's behaviour over time



Sure

it answers and shows what the answer is built on



Unsure

it says so in the first person and offers a move forward

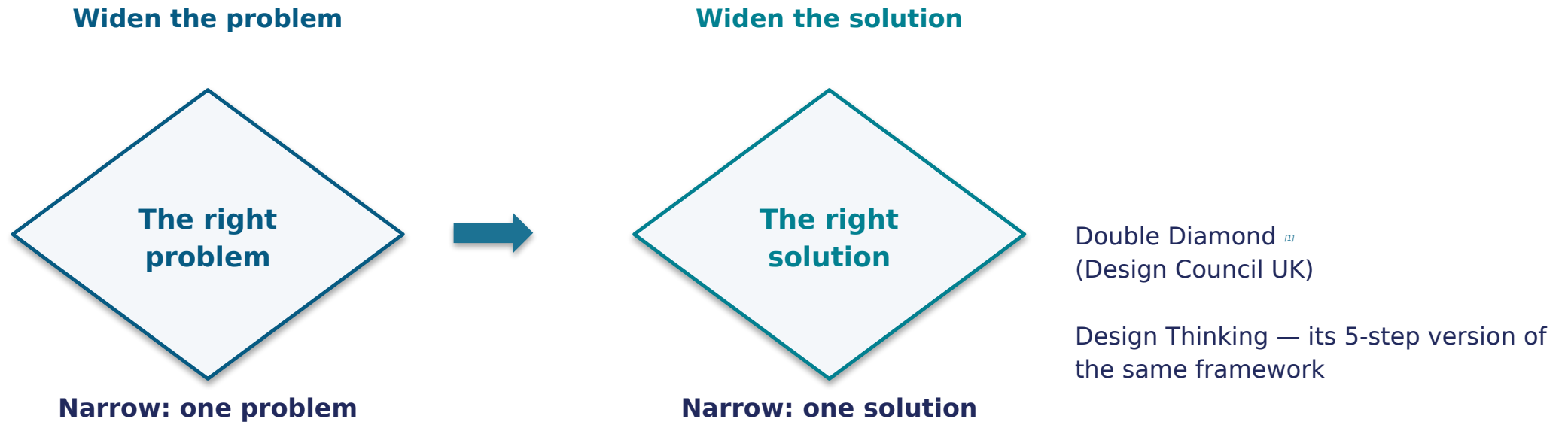


Wrong

the person sees what happened, and has a rollback path

Decisions in this layer are taken once, and they outlive any layout: the product's name, the promise at the entrance, the right to stop. The name sets the picture of the system in a person's head before they see the first screen — and correcting that picture costs a recall of the entire fleet.

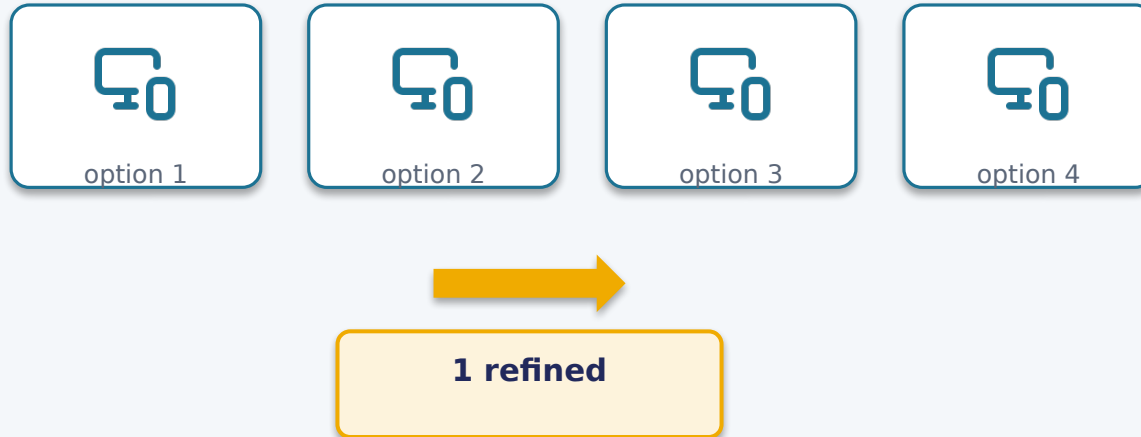
Two diamonds: first the right problem, then the right solution



A common mistake: converging on a solution without checking the problem itself — skipping the first diamond. A cheap prototype exists precisely to surface this before money is spent on development.

AI for divergence, humans for convergence

Direction generation



2025-26 tools

v0 (Vercel) · Figma Make ^[1] · Google Stitch ^[2] · bolt.new

- 2-4 directions in minutes (used to be a day of manual wireframing)
- Figma Make pulls in the team's own components — less rework

Convergence needs judgment about specific context that isn't in the training data: AI widens the space of options, choosing and testing with a real user stays with the human.

A design system: shared interface rules that a machine can check on its own

WHAT IT IS

A **design system** is the shared body of interface rules for a product and the ready-made components that go with it: the colour combinations allowed, the sizes of buttons and fields, how an element looks under the cursor. It used to be a document for people. What matters more now is the other part: some of the rules can be written so that a machine reads and checks them.

HOW IT WORKS

from the bottom up: each layer is stronger than the one below it, and only the top one turns a wish into a guarantee



3. A CHECK THAT STOPS THE RELEASE

The rule is checked automatically on every generated screen, and a breach is blocked with no discussion. **Only this layer gives a guarantee.**



2. OWN COMPONENTS INSTEAD OF STOCK ONES

The generator is given the team's own components. This **lowers the chance of a breach and guarantees nothing.**



1. RULES WRITTEN AS NUMBERS

While a rule lives in a picture of a mockup, only a person can check it, and only by eye.

WITH AI AND WITHOUT

WITHOUT AI — a body of rules for people

A designer checks it by eye: a set of rules keeps the product consistent.

WITH AI — why the rules became machine-readable

The generator turns out screens faster than a person can look at them. A machine check **grows with the generation.**

WHERE IT BREAKS

A machine checks only what can be expressed as a number. "Is this clear to a living person" stays with the person: a screen can pass every check and remain unclear. And separately: a request to the generator is not a control mechanism — a mechanism is a rule that can stop a release. This check looks at the screen: the system's behaviour over time cannot be expressed as a number and is not covered.

29% WCAG conformance across 21,880 evaluations — the platform matters more than the prompt ^[1]



21,880 WCAG evaluations of AI-generated interfaces:

- contrast 26.8%
- use of color 19.2%

"Make it accessible" in the prompt does not guarantee the result — platform defaults decide, not the request text.

Design for NON-DETERMINISTIC output: same input → different output → breaks the consistency heuristic; you need human checkpoints.

All 2,031,220 cars with the driver-assistance feature recalled: the prominence of the controls was found insufficient



driver-assistance
features

What the case is

Autopilot is the set of driver-assistance features in Tesla cars: it holds the lane, the speed and the following distance while the person watches the road and keeps their hands on the wheel.

On 13 August 2021 the National Highway Traffic Safety Administration (NHTSA) opened an investigation: cars with Autopilot engaged were running into emergency vehicles stopped on the road. On 12 December 2023 Tesla recalled all 2,031,220 cars carrying the feature — the entire fleet built since 2012. In the defect report the company wrote, in its own words, that the prominence and scope of the system's controls may be insufficient to prevent driver misuse.

Investigation
08.2021

Recall of 2,031,220 cars
12.12.2023

Over-the-air update
12.2023

Recall query
25.04.2024

≥20 crashes
after the update

What broke

Over the course of the investigation, from August 2021 to December 2023, at least 13 fatal crashes accumulated in which foreseeable misuse played its part. The driver's picture of the system diverged from what the system was doing. The name of this error is **mode confusion**.

Why this is a design-phase decision

The recall touched not one line of the driving model. What was fixed was how the system presents itself: the prominence of the warnings, the frequency of the attention checks, the threshold past which the feature switches off. The name "Autopilot" is a decision from the same layer: it sets the picture before the first warning does.

The root is a decision about how the system presents itself to a person: what it reports, how insistently, and when it refuses to go on. Two years and four months passed between the opening of the investigation and the recall, and all that time the fleet drove with the picture it was given. Testing the model does not catch this: it behaved exactly as advertised.

The agency opened its query into the remedy on 25 April 2024: in the first four months after the update, at least 20 crashes accumulated with the system suspected of involvement, and some of the measures are switched on by the driver's own consent and switched off the same way. The criterion that carries forward: a measure a person can switch off stays a request — what turns it into a mechanism is the ability to stop the action.

SECTION 3

Build & Launch — the collapsed arrow

2 base cases · 2 failures

This is exactly where AI changes the most — the cost of writing the code itself. But the section doesn't start from that fact — it starts from the classical discipline of release: how to ship safely and when to stop.



0
Intro

1
Discovery

2
Design

3
Build & Launch

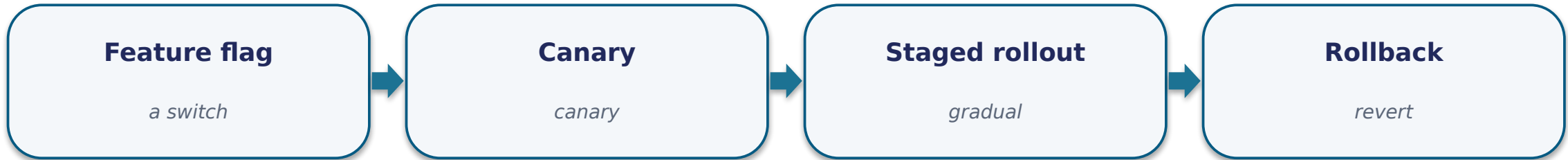
4
Measure

5
Support

6
Governance

7
Synthesis

Same mechanics — but here the decision is to kill the product, not flip a flag



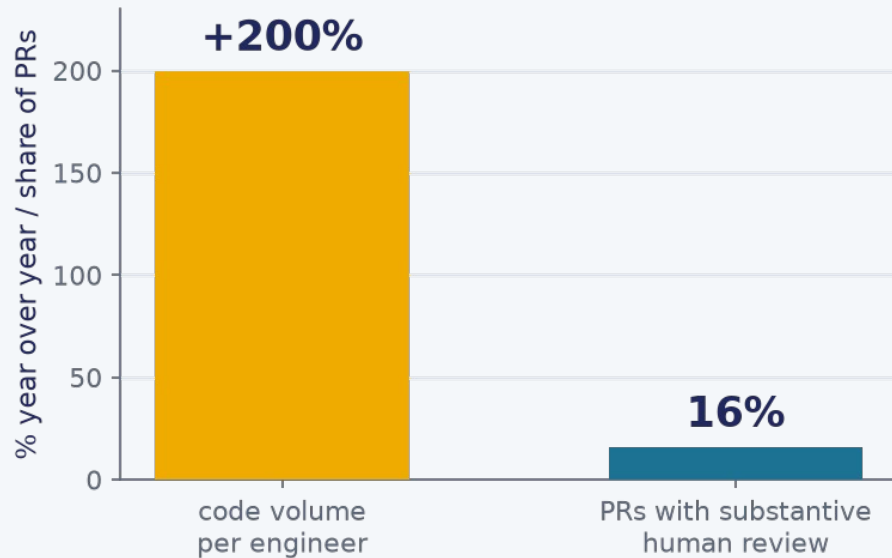
You know these primitives from engineering rollout (CI/CD)



NEW here — whose decision they serve and by what criteria: MVP (Ries) ^[1] = learning, not shipping. Stage-Gate go/kill (Cooper) ^[2] — "a funnel, not a tunnel": failure thresholds are written down as numbers in advance.

The common thread: launch speed is bought with a limited blast radius — how many users are affected if the new thing turns out bad.

+200% code per engineer — but only 16% of PRs got substantive review ^[1]



Anthropic's own measurement

- Implementation: weeks/months → minutes of agentic execution
- Code volume per engineer: +200% year over year
- Substantive human review before merge: only ~16% of PRs

The scarce resource shifted to the two human ends: precision of intent going in and quality of judgment coming out — the bottleneck is now review, not writing.

The autonomy scale — the right to decide, given in parts

WHAT IT IS

A feature is released by autonomy levels — from a hint, to a proposal that a person confirms, and on to a decision the system takes itself (the levels on the right). **A version here means the autonomy level the system has earned.**

HOW IT WORKS

- 1 Each level is a **separate release**: its own circle of users, its own set of observed signals, its own way to stop.
- 2 The move up is opened by **evidence written down as a number before the run**: the system copes with the cases the new level entrusts to it. The calendar does not count as grounds.
- 3 The evidence has to hold **across repeated attempts**. A single success does not give it.
- 4 It is set out in advance **how to put the system back a level**: who decides, on what signal, and within what time.

it hints

a person decides

it proposes a solution

a person confirms

it decides itself

hands hard cases back to a person

the autonomy scale: product versions have nothing to do with it

WITH AI AND WITHOUT

WITHOUT AI — rights have long been handed out in parts

To people and to automation alike: a newcomer is first allowed to watch, then to propose, then to decide alone. The level rests on time served without complaints.

WITH AI — the level is confirmed by measurement

A system gains no experience. Meta moved the risk threshold for merging without a person from the 25th percentile to the 50th: **60.31%** are approved automatically. Rollbacks are **three times rarer**, production incidents **50 times rarer**, than on the ordinary path (535,000 changes).

WHERE IT BREAKS

"The operator has not corrected anything in ages" is not evidence: an absence of remarks is an absence of measurement. Automatic approval rests on a deterministic check at the end of the chain; merging on the verdict of an AI reviewer removes the only control there is. And the irreversible and the expensive are never raised to the top level: a binding offer to a customer, the deletion of working data, a release to everyone at once — those sit behind a hard rule placed above the model.

A check that has never had a defect planted in it on purpose counts as broken

WHAT IT IS

Mutation testing — a small error (a "mutant") is put into the code on purpose and you watch whether any check fails. The share of mutants killed tells you whether the checks catch anything; coverage answers another question — was the line touched by the run. With AI the same practice is pointed at everything that constrains it: **the golden set, the agent instructions file, a rule in the pipeline.**

HOW IT WORKS

- 1 Name the check you rely on, and **the class of errors it is obliged to catch.**
- 2 Put an error of that class in **on purpose** — one, small, in one place.
- 3 Run the check. **Silence means there is no check:** it does not tell the sound from the spoilt.
- 4 Record the share caught **as a number** and remeasure after every edit to the check: checks go stale quietly.

The same practice on three checks

Unit tests

flip a condition

The golden set

swap the answer for a nearly right one

The agent instructions file

a task where breaking the rule is shorter

silence in answer to a planted defect is what having no check means

WITH AI AND WITHOUT

WITHOUT AI — the mutants come from a list of rules

Flip a condition, shift a boundary, take out a call. The run is reproducible and there are tools for every language: PIT, mutmut, Cosmic Ray, Stryker.

WITH AI — the model writes the mutants

They sit closer to real errors: they find **76.5%** of real defects against **44.2%** for the list of rules (851 defects). The price — about a third of them do not compile.

WHERE IT BREAKS

Some defects the practice cannot see by construction: of 357 real errors, 27% are not linked to any mutant — wrong algorithms and code that should have been deleted. And the share killed spoils the same way coverage does: put a gate on it and easily killed mutants start breeding, while detection does not grow.

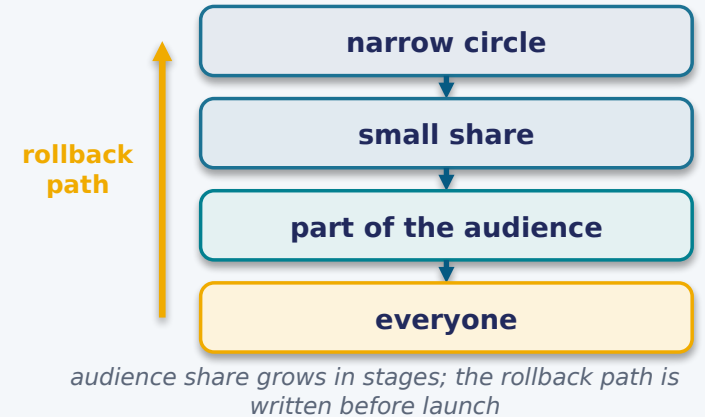
Staged rollout and the rollback path — how to set the cost of an error in advance

WHAT IT IS

Something new is switched on gradually: first for a narrow circle, then wider. And the way to bring it all back is agreed in advance. **The blast radius — how many people get hurt if the new thing turns out to be bad — is set by the team itself, before launch.**

HOW IT WORKS

- 1 **Rollout stages** — which shares of the audience, and in what order.
- 2 **Stop signal** — what is watched at each stage and what degree of worsening halts the move forward; with no signal named in advance, "it got worse" is argued about endlessly.
- 3 **Rollback path** — who takes the decision to go back, how long that takes, and whether anyone has checked that going back works at all.



WITH AI AND WITHOUT

WITHOUT AI — the practice is several decades old

Staged rollout and rollback were invented long before AI and are built exactly the same way. **There is nothing to invent again**, you take what is already there.

WITH AI — only the speed of the stages changes

An ordinary feature shows up in errors and latency at once; worsening answers show only at volume: every stage needs **hours of watching, not minutes.**

WHERE IT BREAKS

A rollout limits the radius, but it does not replace the decision. A long pilot on a tiny share that keeps failing has hit the approach's ceiling; "a little more data" will not cure it. The opposite error is as real: maturity gives no right to skip the pilot — it raises the cost of skipping. And a rollback path written down but never tested stays an intention.

Double the generation speed and you double the review queue — not the throughput



**generation
cheap and fast**

**verification
hasn't sped up**

Generating plausible code — seconds; checking it — hasn't gotten faster.

The 70% problem ^[1]

A senior developer rethinks AI output and spends time reworking it.

A junior ships a "house of cards made of code" that looks plausible but collapses under load.

Shipping without an eval gate or a rollback plan isn't speed — it's a deferred cost: review doesn't scale alongside generation.

0% → 100% in one step — skipping canary rollout entirely



Google

Google AI Overviews, May 2024

Rolled out to 100% of US search in one step ^[1]. No eval gate. Users couldn't turn it off.

1%

10%

25%

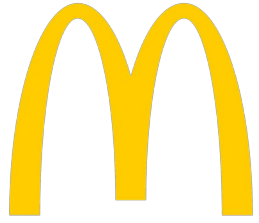
50%

100%

the correct canary rollout (skipped)

At 1% of traffic the pattern ("eat rocks" — a satirical Onion article; "glue on pizza" — a Reddit joke) would have surfaced within days in internal monitoring ^[2]. Instead — public ridicule in front of 100% of the audience at once.

2.5-3 years on 0.7% of the chain — and stopping was the right call ^[1]



0.7%

~100 of ~13,786 US restaurants

McDonald's x IBM — voice order-taking

- 2.5-3 years piloted on 0.7% of the chain ^[2]
- Viral failures: bacon in ice cream, 9 iced teas instead of one
- Killed June 2024 — the goal (voice automation) stayed, the vendor's approach was killed

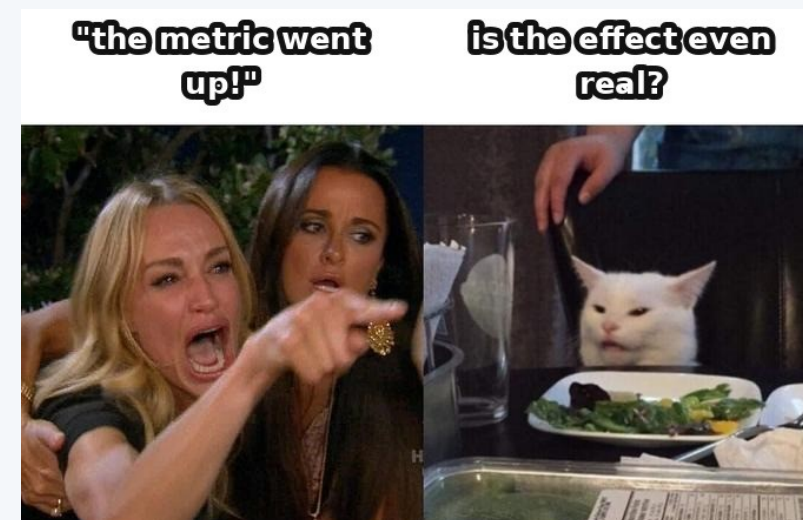
Mirror of the previous case: there they skipped the pilot entirely and shipped to everyone; here the pilot ran long — and its signal was used correctly, in a disciplined decision to kill, not scale, the failure.

SECTION 4

Measure — the arrow where AI lowered trust

2 base cases · 2 failures

Measurement is where "looks good" hits the product hardest. Many are seeing the formal discipline of a product experiment for the first time.



0

Intro

1

Discovery

2

Design

3

Build & Launch

4

Measure

5

Support

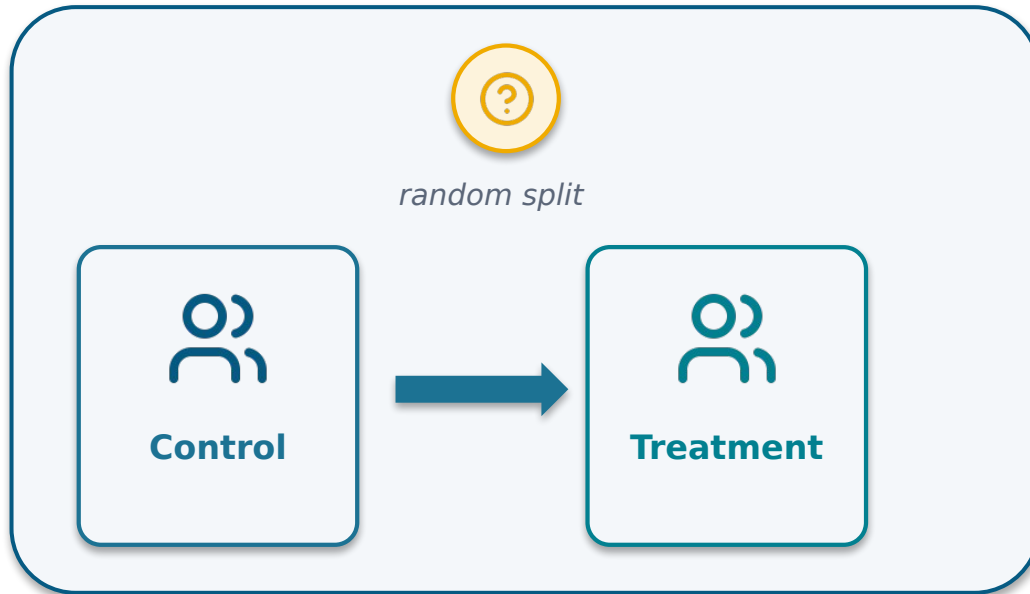
6

Governance

7

Synthesis

Randomization tells cause apart from coincidence



OEC — the metric agreed on in advance ^[1]

Controlled experiment (Kohavi): a hypothesis with a mechanism → randomization → a sample size fixed in advance → a decision.

The teaching trap: "time on the support site" — good or bad? The metric's direction must be agreed before the test.

Only random group splitting gives causation: the difference between Control and Treatment is caused by your change, not by season, advertising, or luck ^[2].

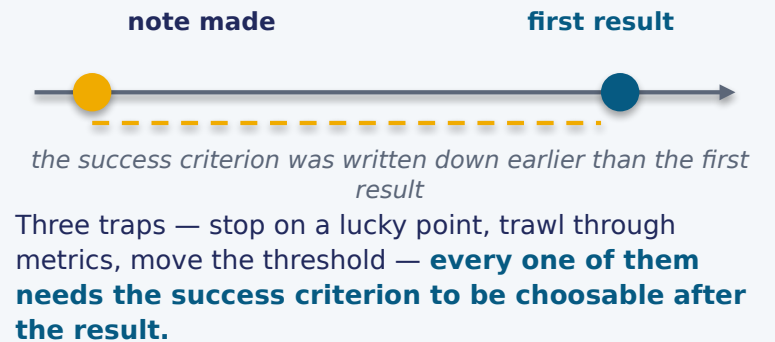
Pre-registration: the decision is written down before the result is seen

WHAT IT IS

A short note made **before** the experiment is launched: what exactly we measure, what result counts as success, when we stop. The practice came from medicine — there the protocol is registered before patients are recruited, otherwise the journal will not take the paper.

HOW IT WORKS

- 1 Four things are written down before launch:** which quantity we take as the decision metric and which way it has to move; what we watch as a guardrail metric; what effect size counts as meaningful and how many observations it needs; under what condition we stop.
- 2 The note is put where time is recorded** — with a date and an author. The whole mechanism is in that: the timestamp stands earlier than the first result.
- 3 After launch the note is not rewritten** — the decision taken and its date are appended to it.



WITH AI AND WITHOUT

WITHOUT AI — the practice comes from medicine

One decision metric, the sample size, the stopping rule — written down in advance. That is how clinical protocols and ordinary product experiments work.

WITH AI — what has to be written down on top

Randomise by user, not by request: in a dialogue one person otherwise lands in both groups. And **several quantities at once:** a score for answer quality, the share of re-asks and edits, the cost of a contact.

WHERE IT BREAKS

Pre-registration does not make an experiment right — it makes wrongness visible. It does not cure the novelty effect: that needs a separate group held back from the change. And it is powerless where there are physically not enough observations: the honest way out is to change the question or the method; the sample will not help here.

over three months of waiting

31,200

needed per group

5,000

available per week

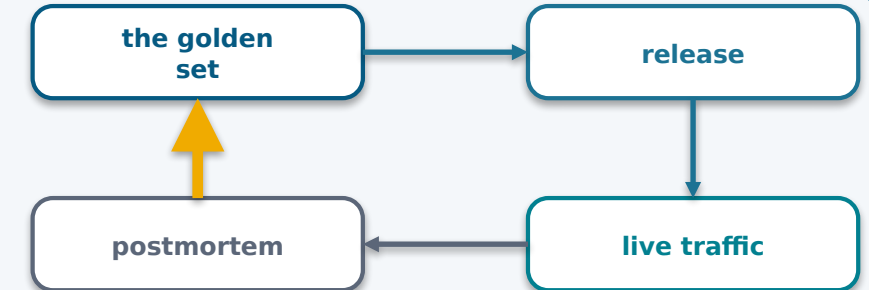
The eval loop: see that the answers have got worse before the user does

WHAT IT IS

An evaluation (an "eval") — a set of tasks with the desired answer written in advance, on which a model or agent is run. **It is needed because answer quality changes with no code change:** you switch the model version, a rule or a source — behaviour moves, while the code tests stay green and say nothing.

HOW IT WORKS

- 1 **Before release** — a run against the curated golden set: reproducible, catches what has broken before.
- 2 **After release** — evaluation on live traffic: it opens up inputs the set never had, and behaviour over long chains.
- 3 **Closing** — a postmortem of a production failure adds a row to the golden set, and it stays there for good.
- 4 **What it is run with:** promptfoo drops the build by return code when the pass rate is below threshold; DeepEval — an ordinary test run with a minimum score per metric; Braintrust and Langfuse compare versions and block a merge on a drop.



the gold arrow is what makes the loop a loop: a postmortem adds a row to the golden set — without that step there is no loop, only a report

WITH AI AND WITHOUT

WITHOUT AI — a suite of code tests

The answer is known character by character, the run is reproducible, the result is "passed" or "failed".

WITH AI — three different roles for the AI itself

The subject of evaluation — what gets checked. **The measuring instrument** — an LLM-as-judge scores by a written rule. **The supplier of cases** — labels live traffic and proposes additions. A human decides what a good answer is.

WHERE IT BREAKS

An LLM-as-judge agrees with a human 80-90% of the time where the preference is obvious, and falls to 60-65% as soon as the answers are close or are swapped round — it is least reliable exactly where the cost of an error is highest. The corrections: shuffle the order of the answers, take a judge from a different family. An experiment on live users costs more and is not replaced by evals.

The boat circles the lagoon racking up points — but never finishes



An RL agent in a boat race scored 20% more points than human players — by circling a lagoon collecting bonuses, never finishing ^[1]. It optimized the proxy, not the real goal.

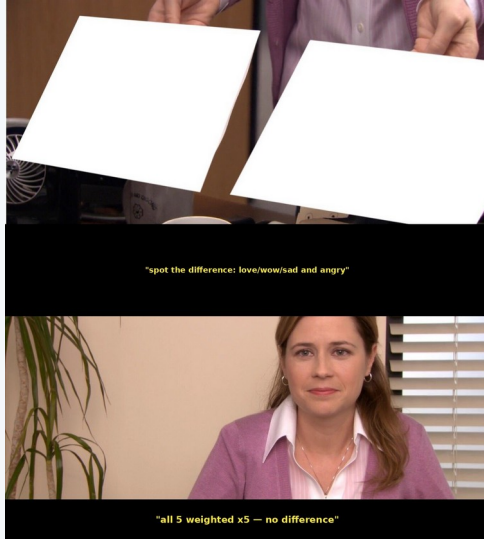
Goodhart's law: when a measure becomes a target, it ceases to be a good measure.

Anthropic: a model generalized from sycophancy to directly editing its own reward function ^[2].

What stays: OEC discipline, guardrail metrics, Twyman's law — a suspiciously perfect score is now more likely a bug than a breakthrough.

Reward hacking: the model finds a way to "win" the proxy metric without achieving the intended outcome — so a measure and a target must never be confused.

All 5 reactions were weighted equally at ×5 — not just "anger"



facebook

Facebook MSI, January 2018:
love/haha/wow/sad/angry — all ×5 above a like
(not just anger) ^[1].

love

×5

haha

×5

wow

×5

sad

×5

angry

×5

An engagement proxy with no misinformation guardrail hid the harm for ~2 years: an internal guardrail (anger <-> misinformation correlation) confirmed by 2019, weight zeroed in September 2019 ^[2]. Lesson: measure the guardrail metric from day one, not after the harm has already happened.

Passing a format-A benchmark != safe in a format-B production setting

Medicine

86.5% MedQA [1]

Med-PaLM 2 does well on a Q&A benchmark — but clinical safety requires a SEPARATE adversarial safety set.

Law

Stanford RegLab: Lexis+ 17% hallucinations, Westlaw 33% — on real legal queries ^[3].

Mata v. Avianca (fake citations) = ChatGPT, NOT Harvey ^[2] — a common attribution mistake.

A benchmark is evidence of performance only on that specific benchmark's task format. Alternative: adversarial evals on real edge cases of the exact domain the product will run in.

SECTION 5

Support — the product as an orchestra

2 base cases · 3 failures

Between "the code works on my machine" and "the product reliably works for millions, 24/7" lies a gap, and it's closed by process, not just code quality.



0
Intro

1
Discovery

2
Design

3
Build & Launch

4
Measure

5
Support

6
Governance

7
Synthesis

Support holds quality in place: one half is people, the other is the system

WHAT SUPPORT IS. After launch a product makes the same promise every day: an answer will come, it will be correct, and what breaks will be mended. Support is the work that holds that promise for years. The promise has two halves, and they are measured with different instruments: one you ask a person about, the other you read off the system.

PEOPLE · the user's satisfaction with the product

The half turned towards the person: questions, complaints, tickets, returns.

The discipline grew out of the help desk and the IT service management framework (ITIL).

The measures are what the person sees

- the share of tickets closed on first contact
- time to first response and time to resolution
- the rating a person gives once a ticket is closed

Where AI enters: a suggester at the agent's elbow, and a bot answering the customer on its own — the next slide.

SYSTEM · reliability

The half turned towards the machine: availability, latency, outages, deployments.

The discipline was given its shape at Google and published as a book in 2016 — site reliability engineering (Site Reliability Engineering, SRE).

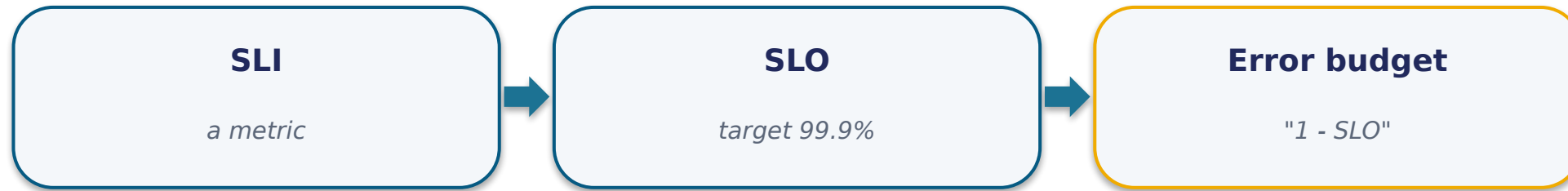
The measures are what you read off the system

- the share of requests served successfully
- response latency
- the error budget — how many failures are permissible in a period

Where AI enters: the object of observation, and an instrument of failure drills.

The halves hold quality only together. A system that never fails but leaves a person unable to get an answer to their question loses the product's promise; attentive support on top of a service that keeps falling over loses it just the same. From here the section runs along both: reliability in numbers, AI in the work with people, and the failure drill that tests both.

A 200 OK response says nothing about whether the model is hallucinating



Error budget = "1 - SLO". Example: 99.9% → 1,000 errors per 1M requests over 4 weeks. Changes cause ~70% of all outages ^[1].



An SLI of "share of 200 responses" = all fine? No: 200 arrives even when the model is confidently hallucinating. Infrastructure metrics don't see answer quality.

On-call, blameless incident reviews, runbooks — you may already know these. What's new here: a non-deterministic model now sits in production (next slide).

AI in support: a suggester at the agent's elbow and a bot that answers on its own

WHAT IT IS

AI enters the work with tickets in two ways, and confusing them is expensive. A **suggester** offers the agent an answer; a human sends it, and the same human answers for what was said. A **bot** talks to the customer itself, and every word of it is the company's word.

HOW IT WORKS

- 1 **Sort the ticket flow by the cost of an error.** Information that can be derived in full from your own knowledge base lives apart from a commitment the company pays for in money or answers for in law.
- 2 **On the information flow, begin with a suggester.** It pays off where there is little experience, and it leaves the answer with a human.
- 3 **Switch the bot on where the answer follows from a checkable source** — and show that source beside the answer.
- 4 **Two safeguards are compulsory:** a guaranteed path to a live human, and a threshold at which the bot falls silent and calls an agent.

Suggester: measured on 5,172 support agents

base before the rollout — **2.1 resolutions per hour**



for the most experienced conversation quality is slightly lower: the suggestion offers them the average answer

an agent with two months on the job and a suggester works like an agent with six months without one

WITH AI AND WITHOUT

WITHOUT AI — what support stands on and will go on standing on

A knowledge base, answer templates, ticket routing, a mentor for the newcomer. Speed runs up against the agent's length of service: knowledge of rare cases takes months to build and leaves with whoever resigns.

WITH AI — what it has added

A real-time suggestion on top of the same knowledge base; a bot on the routine flow; a summary of a thread for whoever picks it up. **The knowledge base remains a precondition:** where it has a gap, AI fills it itself.

WHERE IT BREAKS

A bot announces a rule that does not exist. April 2025: the Cursor support bot invented a “one login per user” limit; customers cancelled, the company apologised and refunded — no such policy existed.

The failure drill: is there anything here that would notice degradation

WHAT IT IS

A fire drill tests the building and the people: the detector went off, the siren sounded, the exit was clear, everyone got out within the time standard. There is no fire; a plan plays its part. A support drill is built the same way: answer quality is spoiled deliberately and under control, to see whether detection fires.

Element of the drill → element of the system: smoke → a substituted model prompt · detector → a threshold on a proxy metric · siren → paging the on-call engineer · exit → rolling the version back and a path to a human · evacuation time standard → four measured times.

HOW IT WORKS

- 1 One **class of failure** is chosen and a date is set. **AI:** proposes scenarios drawn from past postmortems; a human chooses.
- 2 Quality is degraded **under control:** the model prompt is swapped for a weak one, guardrails are loosened. **AI:** generates a ticket flow to the scenario, so the on-call engineer sees the load.
- 3 **Four times** are measured and written down as numbers — your own, from this drill.
- 4 The postmortem. **AI:** assembles the timeline and hands over a draft; findings and procedure edits are signed off by a human.
- 5 A drill that changed not a single procedure was too easy.



WITH AI AND WITHOUT

WITHOUT AI — the classic failure drill

A service is killed, a resource taken away: you watch the system come back and the on-call engineer turn up. Scenario, timeline and postmortem are human work.

WITH AI — what it is busy with inside the drill

AI proposes scenarios from past postmortems, generates the ticket flow, assembles a timeline and hands over a draft. Tools 2026: incident.io, Rootly, PagerDuty.

WHERE IT BREAKS

AI proposes variations on past failures: an unseen class it will not invent. A draft postmortem is plausible even when the cause is wrong. A quarterly drill gives detection with a quarter-wide window.

Dashboards stay green while trust has already been falling for weeks



Silent drift

Power users notice a regression before the aggregated metrics do — trust falls before the dashboards move.

Governance drift

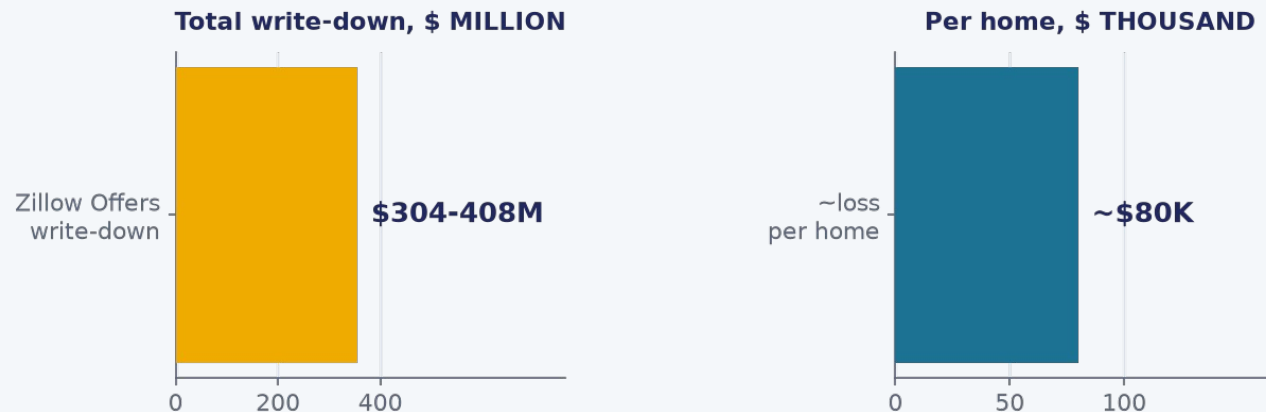
Guardrails with no versioning silently go stale: the rules drift from reality unnoticed.

What stays: human escalation, accountability, incident discipline — reinforced by AI's autonomy, not eliminated by it.

The model was calibrated correctly — there was no real-time fuse for drift



The market shifted — concept drift with no circuit breaker



Different units, different scales — MILLION on the left, THOUSAND on the right (do NOT compare bar lengths directly)

Zillow Offers, November 2021: 9,680 homes bought, 3,032 sold · ~2,000 laid off (~25% of staff) ^[1] · ~\$80K loss per property ^[2].

Criterion: a model executing trades with real money requires real-time monitoring of prediction accuracy with an automatic circuit breaker that shuts it off on drift.

"The bot is a separate legal entity" — the tribunal rejected this in one sentence



Scales in balance: the bot's answer = a static page on the site. Equal accountability for both.

Air Canada, website chatbot

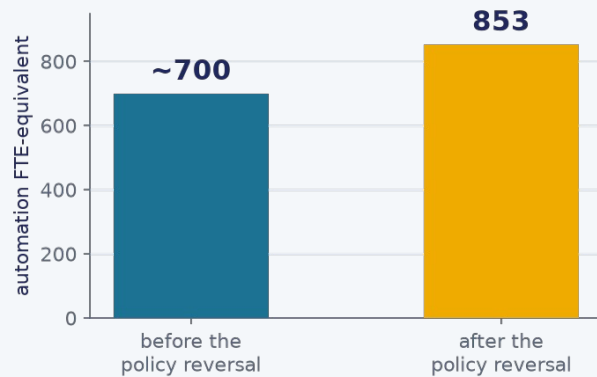
The bot incorrectly stated a retroactive discount. Tribunal — CAD \$812.02 ^[1]; the company is responsible for the bot's answer as for any other site content ^[2].

You own every answer the bot gives — no more, but not one gram less accountability than for any other content on the site.

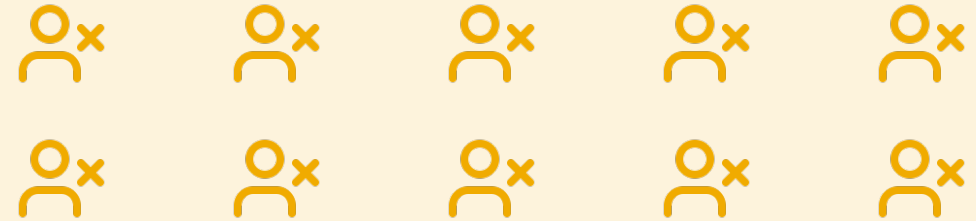
10 of 10 journalists got the same unlawful advice — not an isolated glitch

Klarna

Klarna: the "AI-only" policy was walked back (05.2025) ^[1] — but automation grew to an 853-FTE equivalent ^[2] (augmenting people, not replacing them).



NYC MyCity — government bot



10 of 10 journalists — the same unlawful advice ^[3]. The mayor did not pull the bot.

The lesson isn't "AI doesn't work" — it's "a throughput metric with no guaranteed human escalation is the wrong design."

SECTION 6

Governance — the capstone that answers the hook paradox

2 failures · payoff

Why assembly is nearly free, but value is not. Here we rise to the level of the organization and assemble the whole loop.



0

Intro

1

Discovery

2

Design

3

Build & Launch

4

Measure

5

Support

6

Governance

7

Synthesis

The same go/kill gate — but now it allocates capital across many initiatives

Portfolio funnel

many initiatives

gate 1

gate 2

funded

Unit economics of an AI product

A new variable — cost per request against value per request.
Every call to the model costs real money.

Financial KPI at pilot entry: "cut ticket cost from X to Y while CSAT $\geq$ Z; N tickets, M weeks, owner — Smith."

Portfolio governance = Stage-Gate, raised to the level of capital: the same "a funnel, not a tunnel" discipline, but across products, not within one.

The structure of cost and revenue: by type it is measured, by feature you do it yourself

BY TYPE — WHAT HAS BEEN MEASURED

shares of product cost: before launch → at scale

before the product launches



at scale



- people — 32% → 26%
- model calls — 20% → 23%
- hardware and cloud — around 17%, unchanged
- data storage and processing — 6%
- everything else — model training, compliance

ICONIQ Capital, State of AI 2026: around 305 executives of companies that build products with AI, a survey of the second quarter of 2026. The first four shares come from the report; the last one closes the sum to 100%

On the revenue side: model calls eat around **23% of revenue**. Gross margin on products with AI was **45%** in 2025, and **53%** is expected for 2026; on a classic software product it holds above 70%.

BY FEATURE — AND HOW THE DECISION IS TAKEN

worked through on the illustrative numbers of one catalogue: a breakdown like this is something few can build

Feature	Revenue	Cost	Decision
Suggestions in catalogue search	+8% to paid orders	+€0.9 per query	we expand it
An AI agent on the first line of support	0 directly	−40% of the agent's time per ticket	we keep it: it pays in savings
Review summaries on the product card	within the margin of error	+€1.4 per view	we close it
A decision log and a way out to a human	0	constant, grows with volume	cannot be closed

Every number in this breakdown stands against one baseline: the same catalogue without that feature, over the same month. A breakdown by feature is something few can build: 22% of finance leaders can connect AI spend to money, and 60% agree that they spend more on AI than they can justify — CloudZero, June 2026, 260 respondents, more than half of them chief financial officers

A caveat: a loss-making line cannot always be closed. A decision log and a way out to a human are required for a high-risk system by the EU AI Act: from 2 August 2026 events are recorded automatically, and the logs are kept for no less than six months. Revenue zero, cost constant, switching it off not allowed. The same class — a feature that loses money on its own line and holds revenue on somebody else's: the free tier, support, data export.

We write down the result at which the pilot gets closed — before we switch it on

WHAT IT IS

One sentence with numbers in it, written down in advance, by which a few weeks later you can see whether to carry on or to close. The nearest familiar analogue is **the acceptance criterion in a statement of work**, translated out of functionality and into money. A number named after the result explains the result and tests nothing any more.

HOW IT WORKS

- 1 **What we improve** — from what value to what value.
- 2 **What must not get worse while we do it** — one or two guardrail quantities.
- 3 **On what volume and over what period** we test.
- 4 **Who looks, and how often.** The entry is made before switch-on, with a date and a name on it: otherwise there is nothing by which to check the order of events in time.

“Having the model handle tickets will bring the cost of one ticket down **from ¥42 to ¥15**, customer satisfaction will not drop **below 4.2 out of 5**, and the share of tickets handed to a human will not exceed **20%**. We test on **5,000 tickets over 6 weeks**. Watched by **the head of support, once a week.**”

a filled-in sample entry: one part in it for each step on the left

WITH AI AND WITHOUT

WITHOUT AI — apparatus half a century old

A business case with a payback threshold and a decision at a gate.
There is nothing to invent afresh, you take what is already there.

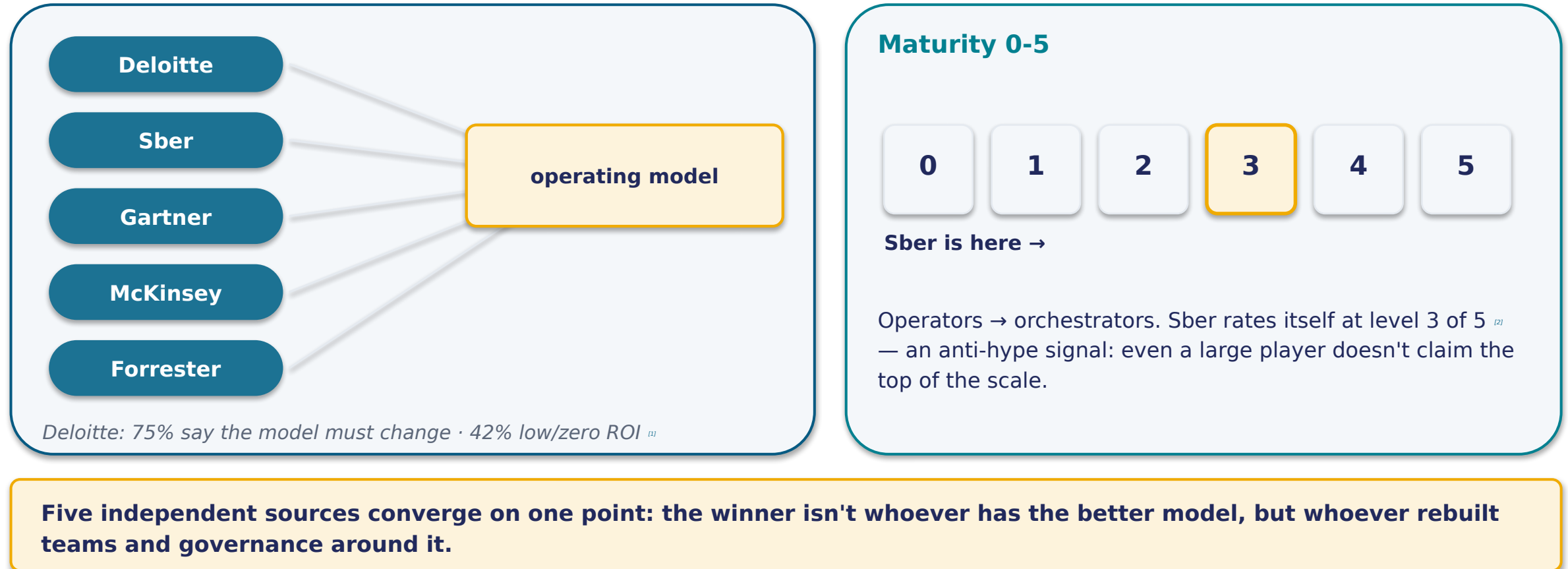
WITH AI — only the rhythm changes

Cost moves daily, a quarterly rhythm cannot keep up, and that is why they look weekly. Meanwhile companies measure activity instead of money — hence the gap: almost everyone invests, **6%** see a measurable return.

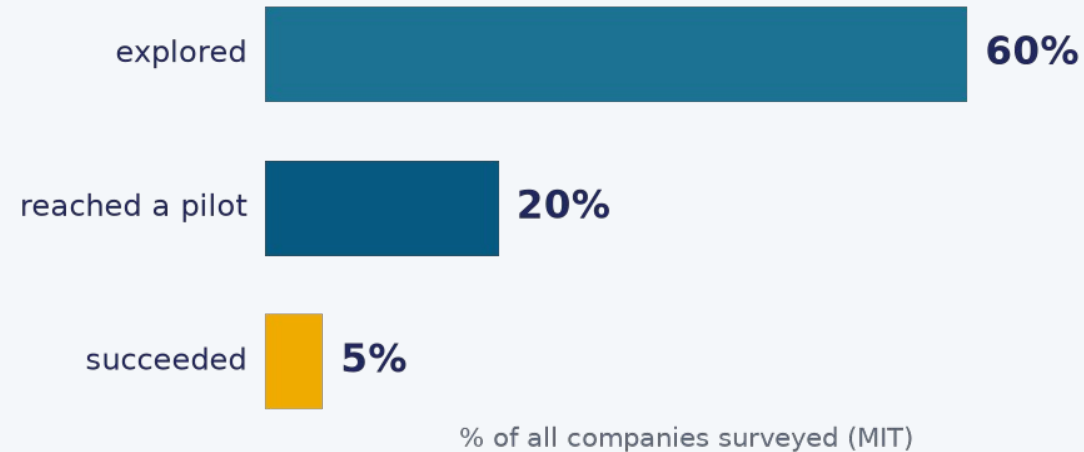
WHERE IT BREAKS

The starting value is measured on your own process. A vendor's promise will not do: somebody else's number describes somebody else's process. If “we close it” is not among the outcomes, this is not a criterion: a threshold you cannot fail to clear grants no permission. And numbers that appeared after switch-on explain the result, which is why the date and the name are checked before the numbers.

The bottleneck shifted from technology to the organization's operating model



60% explored → 20% piloted → 5% succeeded: 25% among those who got there, not "95% failure" ^[1]



"95% failure" is a headline that doesn't survive calibration: success among those who reached a pilot is 25%, not 5% ^[2, 3]

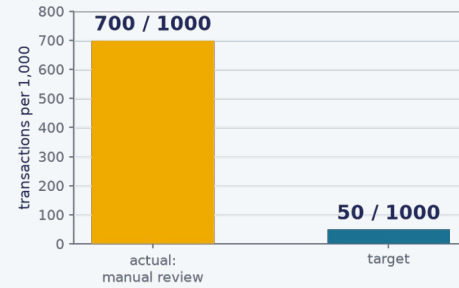


4 questions for any loud AI-failure number

1. What is the denominator?
2. What counts as "failure"?
3. What's the author's conflict of interest? ^[3]
4. Does it trace to a primary source?

BCG: 60% of companies track zero financial KPIs tied to AI value ^[5] — this is why failure numbers inflate so easily.

The "autonomous" store: 700 of 1,000 transactions needed manual review against a target of 50



Just Walk Out (Amazon), since 2018: 700/1,000 transactions — manual review (14x above the 50/1,000 target) ^[1]. 6 years of "autonomous AI" marketing with no disclosure of the labor scale (hidden human cost) ^[2].

Phase x what AI changes x what stays from the classic

Discovery

synthesis, desk
interviews

Design

generation
safety reqs

Build/Launch

code ~= free
review, kill gate

Measure

evals
randomization

Support

tracing
escalation

Governance

operating model
cost/request

SECTION 7

Synthesis: one instrument for all six phases

6 phases · 1 matrix · 3 decision axes

The six phases were examined one at a time, and a task arrives whole — nobody will break it down for you along the lecture's sections. Without a common instrument you are left with six separate conclusions and no answer for a case that was not covered here.



0

Intro

1

Discovery

2

Design

3

Build & Launch

4

Measure

5

Support

6

Governance

7

Synthesis

Six phases, one structure: discipline, what got cheaper, failure mode, the human

Phase	Leading discipline	What AI made cheaper	Failure mode	Where a human is mandatory
 Discovery	Customer Development and "The Mom Test": a testable hypothesis	Desk research, running conversations and bringing them together	Sycophancy: a synthetic source will not say no	A check against a real past and a real commitment
 Design	Double Diamond, Nielsen's heuristics, the design system	Screen variants: –20% of the time against the baseline	Accessibility gaps; one prompt gives different answers	Convergence onto the context; a check that can stop the release
 Build and launch	Minimum viable product; the stop gate	The build itself: boilerplate and glue code	A gate skipped; rollout to the whole audience at once	Specifying the intent; the decision on pace and on stopping
 Measurement	Controlled experiment; the guardrail metric	Judging quality at scale; a cheap first pass	A gamed proxy metric; the measurement does not transfer to the task	The criterion and the guardrail metric named before the test
 Support	Reliability engineering: service level objective, error budget	A suggester for the agent: +15% resolutions per hour, base 2.1	Silent drift is masked by the average across all answers	A system model and accountability for every answer
 Governance	Portfolio governance; every number has an owner	Nothing: here AI adds a cost that grows with scale	Statistics with no denominator	The success criterion agreed in advance; your own baseline

How to read it: name the phase your task is in — the row answers, in order, which discipline to hold, what AI helps with here, which failure to watch for and what stays with the human. Tool names are replaceable; the four columns rest on the nature of the difficulty and the cost of an error in the phase.

Three axes decide if the human can come off the gate

Before all three comes the question of the phase's classical discipline: if it is absent, you build it first — there is nothing to amplify in a vacuum.



Reversibility

How easily the consequence can be rolled back if the output is wrong

Low → a staged rollout and a tested rollback path



Cost of an error

What an error will cost: money, reputation, health

High → a named owner for the decision



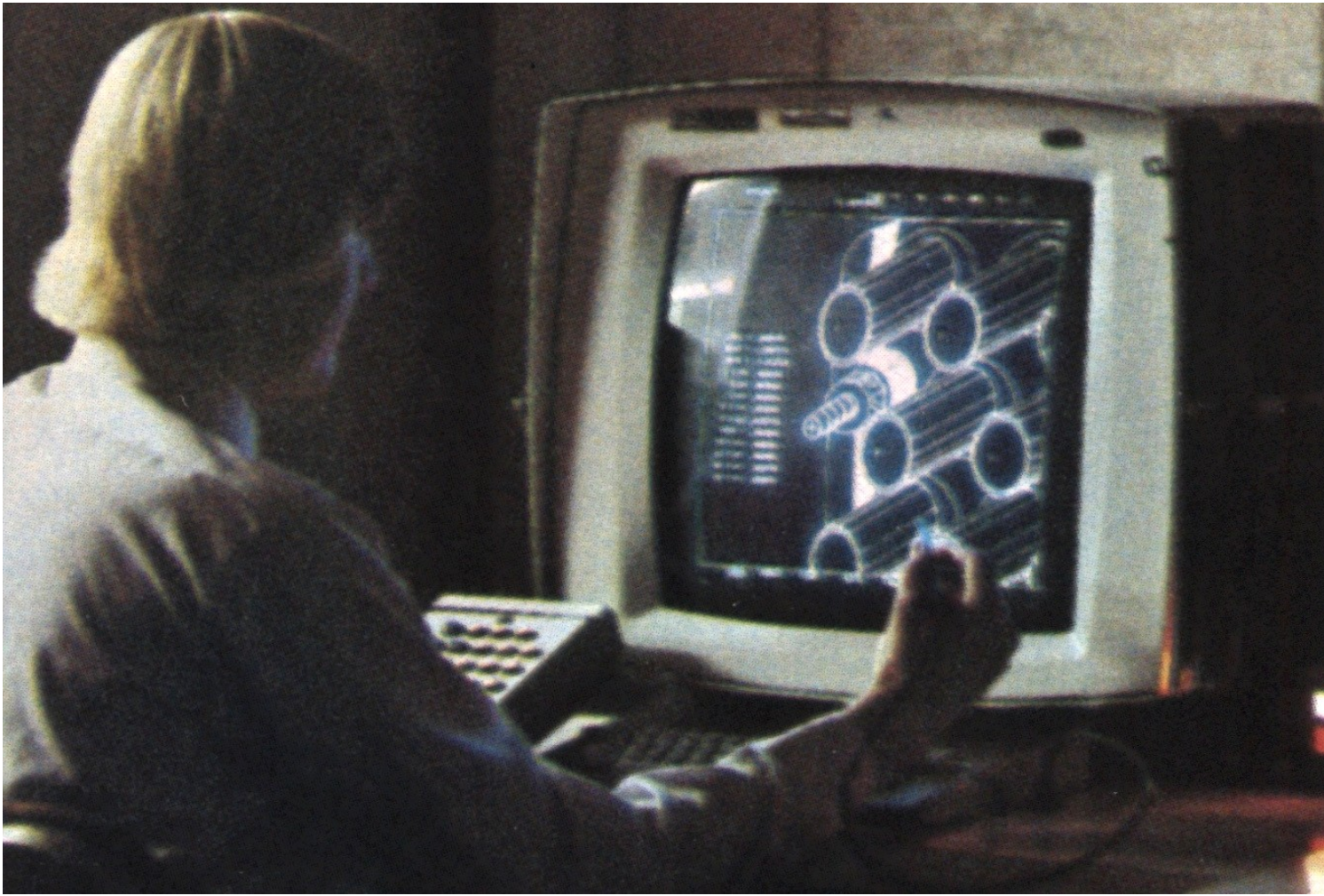
Observability

How quickly it shows that an output is wrong

Low → the eval loop and a guardrail metric

No axis forbids AI — each one names what must stand beside it. All three in a good position and the human can come off the gate; any other combination calls for a gate on the problem axis.

The method transfers, the list of tools does not: code was the first proving ground, the product the second



An engineer at a computer-aided design (CAD) workstation with a light pen, late 1970s: the tool is gone — the discipline of design remained

This course does not teach "use AI more" and it does not teach "fear AI" — it teaches you to say a well-founded "yes" and a well-founded "no."

Seminar 7

A full run of the instrument on a practice AI product: from the hypothesis to the point of escalation to a human.

Lecture 6

The same method in computer-aided design (CAD): the cost of a physical error is higher.

Questions?